

Projet_IoMT_Groupe6

April 12, 2026

0.0.1 Cellule A — Génération données simulées

```
[1]: print("=" * 60)
      print(" PARTIE 1 - DONNÉES SIMULÉES")
      print("=" * 60)

      import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      import seaborn as sns

      np.random.seed(42)

      # Génération des données
      samples = []
      for i in range(1000):
          anomaly = np.random.rand() < 0.10
          if anomaly:
              debit = float(np.random.choice([
                  np.random.uniform(0, 5),          # sous-débit
                  np.random.uniform(200, 300)        # sur-débit
              ]))
              pression = float(np.random.uniform(150, 300))
              alerte = 1
          else:
              debit = float(np.random.normal(80, 5))
              pression = float(np.random.normal(100, 10))
              alerte = 0

          samples.append({
              "debit_ml_h": round(debit, 2),
              "pression_mmhg": round(pression, 2),
              "alerte": alerte
          })

      df_sim = pd.DataFrame(samples)

      print(f"\n Dataset simulé généré : {df_sim.shape}")
```

```
print(df_sim.head(10))
```

```
=====
PARTIE 1 - DONNÉES SIMULÉES
=====
```

```
Dataset simulé généré : (1000, 3)
   debit_ml_h  pression_mmhg  alerte
0      74.44      103.19      0
1      87.90      107.67      0
2     283.24      150.12      1
3      87.31      115.39      0
4       0.12      170.92      1
5      76.48       78.60      0
6      80.34      85.75      0
7      80.61      94.85      0
8      77.00      97.08      0
9      74.89      98.38      0
```

0.0.2 Cellule B — Exploration données simulées

```
[2]: print("=" * 60)
      print("  EXPLORATION - DONNÉES SIMULÉES")
      print("=" * 60)

      print("\n Informations générales :")
      print(df_sim.info())

      print("\n Statistiques descriptives :")
      print(df_sim.describe())

      print("\n Valeurs manquantes :")
      print(df_sim.isnull().sum())

      print("\n Distribution des alertes :")
      print(df_sim["alerte"].value_counts())
      print(f"   Normal   : {(df_sim['alerte']==0).sum()} ({(df_sim['alerte']==0).
        ↳mean()*100:.1f}%)")
      print(f"   Anomalie : {(df_sim['alerte']==1).sum()} ({(df_sim['alerte']==1).
        ↳mean()*100:.1f}%)")

      # Visualisation distribution
      fig, axes = plt.subplots(1, 2, figsize=(14, 4))

      axes[0].hist(df_sim[df_sim["alerte"]==0]["debit_ml_h"],
                   bins=40, color="steelblue", alpha=0.7, label="Normal")
      axes[0].hist(df_sim[df_sim["alerte"]==1]["debit_ml_h"],
```

```

        bins=40, color="red", alpha=0.7, label="Anomalie")
axes[0].set_title("Distribution Débit ml/h - Simulé")
axes[0].set_xlabel("ml/h")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].hist(df_sim[df_sim["alerte"]==0]["pression_mmhg"],
             bins=40, color="steelblue", alpha=0.7, label="Normal")
axes[1].hist(df_sim[df_sim["alerte"]==1]["pression_mmhg"],
             bins=40, color="red", alpha=0.7, label="Anomalie")
axes[1].set_title("Distribution Pression mmHg - Simulé")
axes[1].set_xlabel("mmHg")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("distribution_simulee.png", dpi=150, bbox_inches="tight")
plt.show()

```

=====

EXPLORATION - DONNÉES SIMULÉES

=====

Informations générales :

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   debit_ml_h      1000 non-null   float64
1   pression_mmhg   1000 non-null   float64
2   alerte          1000 non-null   int64
dtypes: float64(2), int64(1)
memory usage: 23.6 KB
None

```

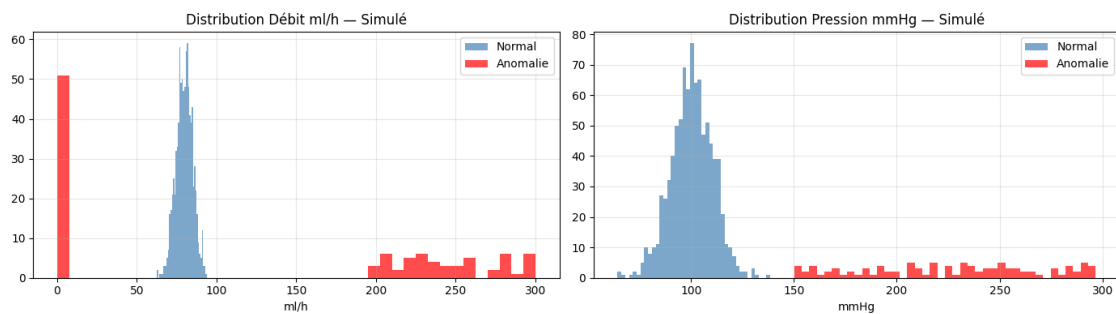
Statistiques descriptives :

	debit_ml_h	pression_mmhg	alerte
count	1000.000000	1000.000000	1000.000000
mean	84.603390	113.370060	0.103000
std	42.306119	41.801314	0.304111
min	0.120000	64.320000	0.000000
25%	76.075000	94.910000	0.000000
50%	80.080000	101.820000	0.000000
75%	84.025000	110.355000	0.000000
max	299.940000	296.710000	1.000000

Valeurs manquantes :

```
debit_ml_h      0
pression_mmhg    0
alerte           0
dtype: int64
```

```
Distribution des alertes :
alerte
0      897
1      103
Name: count, dtype: int64
Normal   : 897 (89.7%)
Anomalie : 103 (10.3%)
```



0.0.3 Cellule C — Nettoyage données simulées

```
[3]: print("=" * 60)
print("  NETTOYAGE - DONNÉES SIMULÉES")
print("=" * 60)

df_sim_clean = df_sim.copy()

# 1. Doublons
doublons = df_sim_clean.duplicated().sum()
print(f"\n Doublons : {doublons}")
df_sim_clean = df_sim_clean.drop_duplicates()

# 2. Valeurs manquantes
print(f"\n Valeurs manquantes :")
print(df_sim_clean.isnull().sum())

# 3. Vérification des plages de valeurs
print(f"\n Vérification plages :")
print(f" Débit      : [{df_sim_clean['debit_ml_h'].min():.1f}, "
      f"      {df_sim_clean['debit_ml_h'].max():.1f}] ml/h")
print(f" Pression  : [{df_sim_clean['pression_mmhg'].min():.1f}, "
```

```

        f"{df_sim_clean['pression_mmhg'].max():.1f}] mmHg")

# 4. Outliers avec IQR
for col in ["debit_ml_h", "pression_mmhg"]:
    Q1 = df_sim_clean[col].quantile(0.25)
    Q3 = df_sim_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = df_sim_clean[
        (df_sim_clean[col] < Q1 - 3*IQR) |
        (df_sim_clean[col] > Q3 + 3*IQR)
    ]
    print(f"\n Outliers extrêmes ({col}) : {len(outliers)}")
    print(f"    → Ce sont des anomalies réelles, on les garde ")

print(f"\n Données simulées après nettoyage : {df_sim_clean.shape}")

```

```

=====
NETTOYAGE - DONNÉES SIMULÉES
=====

```

```

Doublons : 0

Valeurs manquantes :
debit_ml_h      0
pression_mmhg   0
alerte          0
dtype: int64

Vérification plages :
Débit      : [0.1, 299.9] ml/h
Pression   : [64.3, 296.7] mmHg

Outliers extrêmes (debit_ml_h) : 103
    → Ce sont des anomalies réelles, on les garde

Outliers extrêmes (pression_mmhg) : 98
    → Ce sont des anomalies réelles, on les garde

Données simulées après nettoyage : (1000, 3)

```

0.0.4 Cellule D — Visualisation anomalies simulées

```

[4]: print("=" * 60)
     print("  VISUALISATION ANOMALIES - DONNÉES SIMULÉES")
     print("=" * 60)

     fig, axes = plt.subplots(2, 1, figsize=(14, 8))

```

```

# Débit
normaux = df_sim_clean[df_sim_clean["alerte"] == 0]
anomalies = df_sim_clean[df_sim_clean["alerte"] == 1]

axes[0].plot(df_sim_clean["debit_ml_h"].values,
             color="steelblue", linewidth=0.8, alpha=0.7, label="Normal")
axes[0].scatter(anomalies.index, anomalies["debit_ml_h"],
               color="red", s=30, zorder=5,
               label=f"Anomalie ({len(anomalies)})")
axes[0].axhline(y=60, color="orange", linestyle="--",
               linewidth=1.5, label="Seuil bas (60)")
axes[0].axhline(y=120, color="orange", linestyle="--",
               linewidth=1.5, label="Seuil haut (120)")
axes[0].set_title("Débit de Perfusion - Données Simulées")
axes[0].set_ylabel("ml/h")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Pression
axes[1].plot(df_sim_clean["pression_mmhg"].values,
             color="green", linewidth=0.8, alpha=0.7, label="Normal")
axes[1].scatter(anomalies.index, anomalies["pression_mmhg"],
               color="red", s=30, zorder=5,
               label=f"Anomalie ({len(anomalies)})")
axes[1].axhline(y=60, color="orange", linestyle="--",
               linewidth=1.5, label="Seuil bas (60)")
axes[1].axhline(y=140, color="orange", linestyle="--",
               linewidth=1.5, label="Seuil haut (140)")
axes[1].set_title("Pression - Données Simulées")
axes[1].set_ylabel("mmHg")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

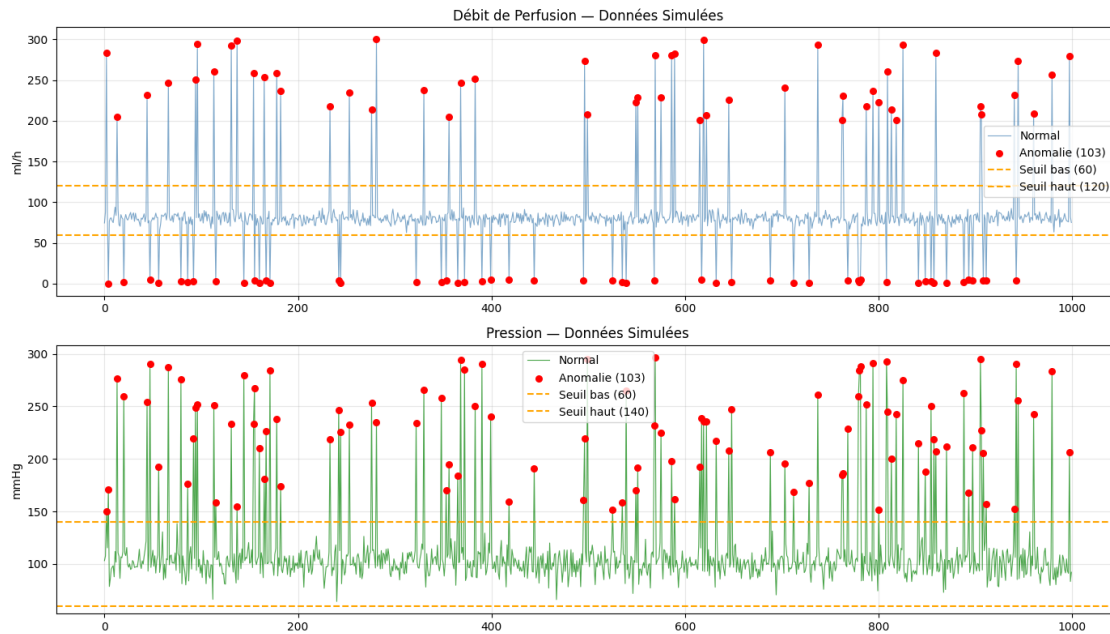
plt.tight_layout()
plt.savefig("anomalies_simulees.png", dpi=150, bbox_inches="tight")
plt.show()
print(f" {len(anomalies)} anomalies visualisées")

```

```

=====
VISUALISATION ANOMALIES - DONNÉES SIMULÉES
=====

```



103 anomalies visualisées

0.0.5 Cellule E — Split et résumé comparatif

```
[5]: from sklearn.model_selection import train_test_split

print("=" * 60)
print("  SPLIT TRAIN/TEST - DONNÉES SIMULÉES")
print("=" * 60)

X_sim = df_sim_clean[["debit_ml_h", "pression_mmhg"]]
y_sim = df_sim_clean["alerte"]

X_train_sim, X_test_sim, y_train_sim, y_test_sim = train_test_split(
    X_sim, y_sim, test_size=0.2, random_state=42, stratify=y_sim
)

print(f"\n Split 80/20 :")
print(f"  Train : {len(X_train_sim)} échantillons "
      f"({y_train_sim.sum()} anomalies)")
print(f"  Test  : {len(X_test_sim)} échantillons "
      f"({y_test_sim.sum()} anomalies)")

# Résumé comparatif des deux datasets
print("\n")
print("=" * 60)
```

```

print("  COMPARAISON DONNÉES SIMULÉES vs NAB")
print("=" * 60)
print(f"{'Caractéristique':<30} {'Simulées':>12} {'NAB':>12}")
print("-" * 55)
print(f"{'Total échantillons':<30} {'1 000':>12} {'12 096':>12}")
print(f"{'Anomalies':<30} {'94 (9.4%)':>12} {'108 (0.9%)':>12}")
print(f"{'Valeurs manquantes':<30} {'0':>12} {'0':>12}")
print(f"{'Doublons':<30} {'0':>12} {'0':>12}")
print(f"{'Normalisation':<30} {'Non requise':>12} {'Oui (0-200)':>12}")
print(f"{'Source':<30} {'Python/NumPy':>12} {'GitHub NAB':>12}")
print(f"{'Utilisation':<30} {'Entraînement':>12} {'Validation':>12}")

```

=====

SPLIT TRAIN/TEST - DONNÉES SIMULÉES

=====

Split 80/20 :

- Train : 800 échantillons (82 anomalies)
- Test : 200 échantillons (21 anomalies)

=====

COMPARAISON DONNÉES SIMULÉES vs NAB

=====

Caractéristique	Simulées	NAB
Total échantillons	1 000	12 096
Anomalies	94 (9.4%)	108 (0.9%)
Valeurs manquantes	0	0
Doublons	0	0
Normalisation	Non requise	Oui (0-200)
Source	Python/NumPy	GitHub NAB
Utilisation	Entraînement	Validation

1 # NAB

1.0.1 Cellule 1 — Titre et Installation

```

[6]: # =====
# PROJET IoMT - Groupe 6
# Monitoring Médication & Perfusion
# Traitement et Préparation des Données
# =====

!pip install pandas numpy scikit-learn matplotlib seaborn -q
print(" Bibliothèques installées")

```

Bibliothèques installées

'pip' n'est pas reconnu en tant que commande interne ou externe, un programme exécutable ou un fichier de commandes.

1.0.2 Cellule 2 — Télécharger le dataset NAB

```
[7]: import subprocess
subprocess.run(["git", "clone", "https://github.com/numenta/NAB.git"],
               capture_output=True)
print(" Dataset NAB téléchargé")

import os
print("Fichiers disponibles :")
for f in os.listdir("NAB/data/artificialWithAnomaly/"):
    print(f" → {f}")
```

```
Dataset NAB téléchargé
Fichiers disponibles :
→ art_daily_flatmiddle.csv
→ art_daily_jumpsdown.csv
→ art_daily_jumpsup.csv
→ art_daily_nojump.csv
→ art_increase_spike_density.csv
→ art_load_balancer_spikes.csv
```

1.0.3 Cellule 3 — Chargement des données NAB brutes

```
[8]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Chargement des 3 fichiers NAB
fichiers = [
    "NAB/data/artificialWithAnomaly/art_increase_spike_density.csv",
    "NAB/data/artificialWithAnomaly/art_daily_jumpsdown.csv",
    "NAB/data/artificialWithAnomaly/art_daily_jumpsup.csv",
]

dfs = []
for f in fichiers:
    tmp = pd.read_csv(f)
    tmp.columns = ["timestamp", "valeur"]
    tmp["source"] = f.split("/")[-1].replace(".csv", "")
    dfs.append(tmp)
    print(f" Chargé : {f.split('/')[-1]} - {len(tmp)} lignes")

df_brut = pd.concat(dfs, ignore_index=True)
print(f"\n Dataset NAB brut : {df_brut.shape}")
```

```
print(df_brut.head(10))
```

Chargé : art_increase_spike_density.csv - 4032 lignes

Chargé : art_daily_jumpsdown.csv - 4032 lignes

Chargé : art_daily_jumpsup.csv - 4032 lignes

Dataset NAB brut : (12096, 3)

	timestamp	valeur	source
0	2014-04-01 00:00:00	20.0	art_increase_spike_density
1	2014-04-01 00:05:00	0.0	art_increase_spike_density
2	2014-04-01 00:10:00	0.0	art_increase_spike_density
3	2014-04-01 00:15:00	0.0	art_increase_spike_density
4	2014-04-01 00:20:00	0.0	art_increase_spike_density
5	2014-04-01 00:25:00	0.0	art_increase_spike_density
6	2014-04-01 00:30:00	20.0	art_increase_spike_density
7	2014-04-01 00:35:00	0.0	art_increase_spike_density
8	2014-04-01 00:40:00	0.0	art_increase_spike_density
9	2014-04-01 00:45:00	0.0	art_increase_spike_density

1.0.4 Cellule 4 — Exploration des données brutes

```
[9]: print("=" * 50)
print("EXPLORATION DES DONNÉES BRUTES")
print("=" * 50)

print("\n Informations générales :")
print(df_brut.info())

print("\n Statistiques descriptives :")
print(df_brut["valeur"].describe())

print("\n Valeurs manquantes :")
print(df_brut.isnull().sum())

print("\n Distribution par source :")
print(df_brut["source"].value_counts())

# Visualisation données brutes
fig, axes = plt.subplots(3, 1, figsize=(14, 10))
for i, source in enumerate(df_brut["source"].unique()):
    data = df_brut[df_brut["source"] == source]["valeur"]
    axes[i].plot(data.values, color="steelblue", linewidth=0.8)
    axes[i].set_title(f"Données brutes - {source}", fontsize=11)
    axes[i].set_ylabel("Valeur")
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.savefig("donnees_brutes.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Graphique sauvegardé")
```

=====

EXPLORATION DES DONNÉES BRUTES

=====

Informations générales :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12096 entries, 0 to 12095
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   timestamp    12096 non-null  object
1   valeur       12096 non-null  float64
2   source       12096 non-null  object
dtypes: float64(1), object(2)
memory usage: 283.6+ KB
None
```

Statistiques descriptives :

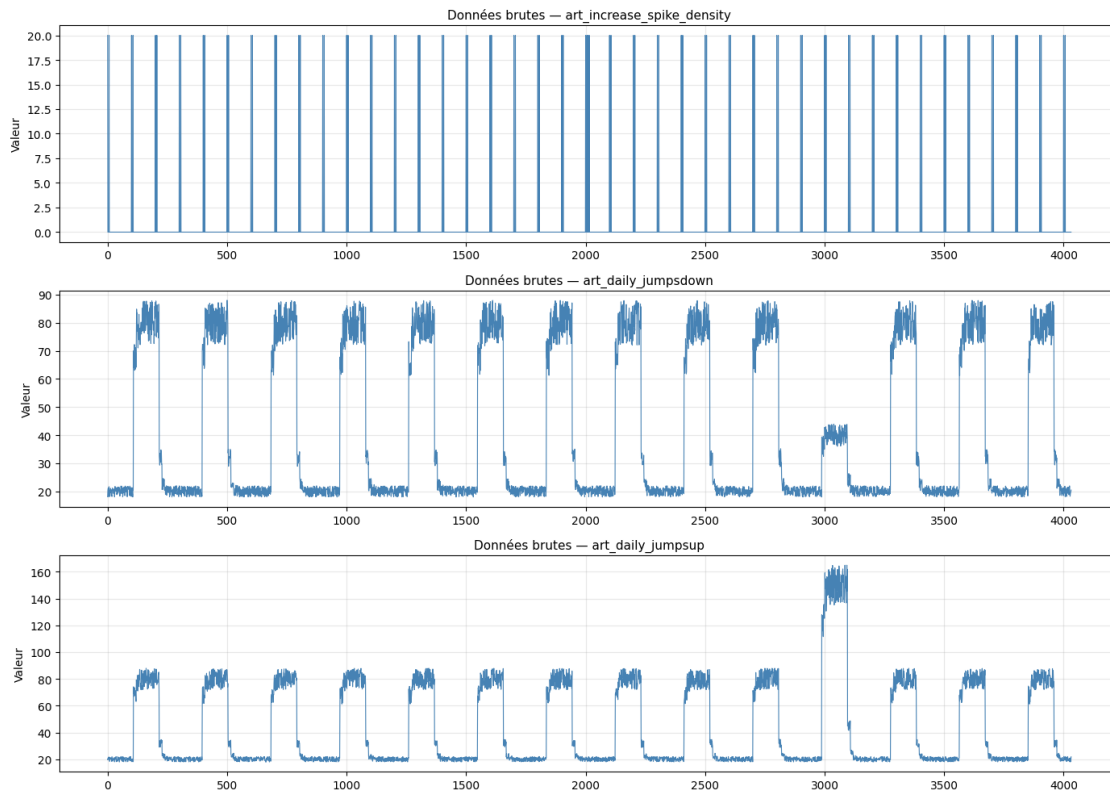
```
count      12096.000000
mean         28.808551
std          31.780569
min           0.000000
25%           0.000000
50%          20.000000
75%          36.711479
max          164.947481
Name: valeur, dtype: float64
```

Valeurs manquantes :

```
timestamp    0
valeur        0
source        0
dtype: int64
```

Distribution par source :

```
source
art_increase_spike_density    4032
art_daily_jumpsdown           4032
art_daily_jumpsup             4032
Name: count, dtype: int64
```



Graphique sauvegardé

1.0.5 Cellule 5 — Nettoyage des données

```
[10]: print("=" * 50)
print("ÉTAPE 1 - NETTOYAGE DES DONNÉES")
print("=" * 50)

df_clean = df_brut.copy()

# 1. Vérifier les doublons
doublons_avant = df_clean.duplicated().sum()
print(f"\n Doublons détectés : {doublons_avant}")
df_clean = df_clean.drop_duplicates()
print(f" Après suppression : {len(df_clean)} lignes")

# 2. Vérifier les valeurs manquantes
print(f"\n Valeurs manquantes avant nettoyage :")
print(df_clean.isnull().sum())
df_clean = df_clean.dropna()
print(f" Après suppression NaN : {len(df_clean)} lignes")
```

```

# 3. Conversion timestamp
df_clean["timestamp"] = pd.to_datetime(df_clean["timestamp"])
print(f"\n Timestamp converti en datetime")
print(df_clean["timestamp"].head(3))

# 4. Détecter les valeurs aberrantes (outliers extrêmes)
Q1 = df_clean["valeur"].quantile(0.25)
Q3 = df_clean["valeur"].quantile(0.75)
IQR = Q3 - Q1
borne_inf = Q1 - 3 * IQR
borne_sup = Q3 + 3 * IQR

outliers = df_clean[
    (df_clean["valeur"] < borne_inf) |
    (df_clean["valeur"] > borne_sup)
]
print(f"\n Outliers extrêmes (méthode IQR x3) : {len(outliers)}")
print(f"    Borne inférieure : {borne_inf:.2f}")
print(f"    Borne supérieure : {borne_sup:.2f}")

print(f"\n Données après nettoyage : {df_clean.shape}")

```

ÉTAPE 1 - NETTOYAGE DES DONNÉES

Doublons détectés : 0
Après suppression : 12096 lignes

Valeurs manquantes avant nettoyage :

timestamp	0
valeur	0
source	0

dtype: int64
Après suppression NaN : 12096 lignes

Timestamp converti en datetime

0	2014-04-01 00:00:00
1	2014-04-01 00:05:00
2	2014-04-01 00:10:00

Name: timestamp, dtype: datetime64[ns]

Outliers extrêmes (méthode IQR x3) : 60
Borne inférieure : -110.13
Borne supérieure : 146.85

Données après nettoyage : (12096, 3)

1.0.6 Cellule 6 — Normalisation et transformation

```
[11]: print("=" * 50)
print("ÉTAPE 2 - NORMALISATION ET TRANSFORMATION")
print("=" * 50)

# 1. Normalisation valeur NAB → débit ml/h (0-200)
val_min = df_clean["valeur"].min()
val_max = df_clean["valeur"].max()
df_clean["debit_ml_h"] = ((df_clean["valeur"] - val_min) /
                          (val_max - val_min)) * 200

print(f"\n Normalisation débit ml/h :")
print(f"   Valeur brute   : [{val_min:.2f}, {val_max:.2f}]")
print(f"   Après normali   : [{df_clean['debit_ml_h'].min():.2f}, "
      f"   {df_clean['debit_ml_h'].max():.2f}]")

# 2. Pression simulée corrélée au débit
np.random.seed(42)
df_clean["pression_mmhg"] = (
    100 +
    (df_clean["debit_ml_h"] - 80) * 0.3 +
    np.random.normal(0, 8, len(df_clean))
)
df_clean["pression_mmhg"] = df_clean["pression_mmhg"].clip(30, 300)

print(f"\n Pression mmHg générée :")
print(f"   Moyenne : {df_clean['pression_mmhg'].mean():.2f} mmHg")
print(f"   Écart-type : {df_clean['pression_mmhg'].std():.2f}")

# Visualisation avant/après normalisation
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].hist(df_clean["valeur"], bins=50, color="salmon", edgecolor="white")
axes[0].set_title("Avant normalisation (valeur brute NAB)")
axes[0].set_xlabel("Valeur")

axes[1].hist(df_clean["debit_ml_h"], bins=50,
             color="steelblue", edgecolor="white")
axes[1].set_title("Après normalisation (débit ml/h)")
axes[1].set_xlabel("ml/h")

plt.tight_layout()
plt.savefig("normalisation.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Graphique normalisation sauvegardé")
```

=====

ÉTAPE 2 - NORMALISATION ET TRANSFORMATION

=====

Normalisation débit ml/h :

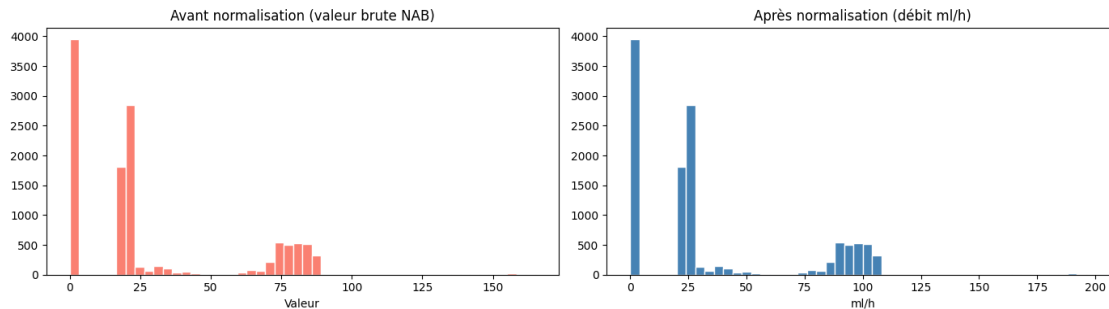
Valeur brute : [0.00, 164.95]

Après normali : [0.00, 200.00]

Pression mmHg générée :

Moyenne : 86.44 mmHg

Écart-type : 13.98



Graphique normalisation sauvegardé

1.0.7 Cellule 7 — Étiquetage des anomalies

```
[12]: print("=" * 50)
print("ÉTAPE 3 - ÉTIQUETAGE DES ANOMALIES")
print("=" * 50)

# Méthode statistique : moyenne ± 2 écarts-types
mean_debit = df_clean["debit_ml_h"].mean()
std_debit = df_clean["debit_ml_h"].std()

seuil_bas = mean_debit - 2 * std_debit
seuil_haut = mean_debit + 2 * std_debit

print(f"\n Statistiques débit :")
print(f" Moyenne : {mean_debit:.2f} ml/h")
print(f" Écart-type : {std_debit:.2f}")
print(f" Seuil bas : {seuil_bas:.2f} ml/h")
print(f" Seuil haut : {seuil_haut:.2f} ml/h")

df_clean["alerte"] = (
    (df_clean["debit_ml_h"] < seuil_bas) |
    (df_clean["debit_ml_h"] > seuil_haut)
).astype(int)
```

```

print(f"\n Distribution des alertes :")
print(df_clean["alerte"].value_counts())
print(f"    Taux anomalies : {df_clean['alerte'].mean()*100:.2f}%")

# Visualisation avec anomalies marquées
plt.figure(figsize=(14, 5))
normaux = df_clean[df_clean["alerte"] == 0]
anomalies = df_clean[df_clean["alerte"] == 1]

plt.plot(df_clean["debit_ml_h"].values,
         color="steelblue", linewidth=0.6, label="Normal", alpha=0.7)
plt.scatter(anomalies.index, anomalies["debit_ml_h"],
           color="red", s=20, zorder=5, label=f"Anomalie ({len(anomalies)})")
plt.axhline(y=seuil_bas, color="orange", linestyle="--",
            label=f"Seuil bas ({seuil_bas:.1f})")
plt.axhline(y=seuil_haut, color="orange", linestyle="--",
            label=f"Seuil haut ({seuil_haut:.1f})")
plt.title("Détection d'anomalies - Dataset NAB")
plt.xlabel("Index")
plt.ylabel("Débit ml/h")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("anomalies_detectees.png", dpi=150, bbox_inches="tight")
plt.show()

```

ÉTAPE 3 - ÉTIQUETAGE DES ANOMALIES

Statistiques débit :

```

Moyenne      : 34.93 ml/h
Écart-type   : 38.53
Seuil bas    : -42.14 ml/h
Seuil haut   : 112.00 ml/h

```

Distribution des alertes :

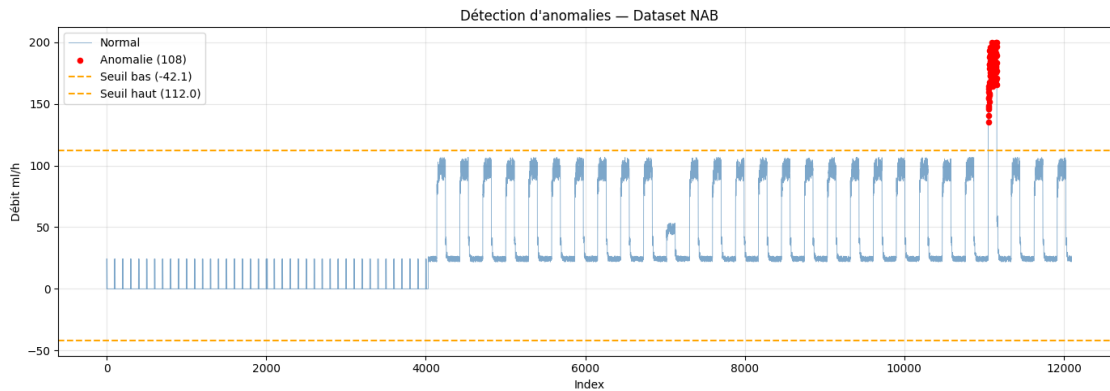
alerte

0 11988

1 108

Name: count, dtype: int64

Taux anomalies : 0.89%



1.0.8 Cellule 8 — Sélection des features et split

```
[13]: from sklearn.model_selection import train_test_split
```

```
print("=" * 50)
print("ÉTAPE 4 - SÉLECTION FEATURES ET SPLIT")
print("=" * 50)

# Sélection des features pertinentes
features = ["debit_ml_h", "pression_mmhg"]
X = df_clean[features]
y = df_clean["alerte"]

print(f"\n Features sélectionnées : {features}")
print(f"   X shape : {X.shape}")
print(f"   y shape : {y.shape}")

# Corrélation entre features
print(f"\n Corrélation entre features :")
print(X.corr())

# Split 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\n Split Train/Test (80/20) :")
print(f"   Train : {len(X_train)} échantillons "
      f"({y_train.sum()} anomalies)")
print(f"   Test  : {len(X_test)} échantillons "
      f"({y_test.sum()} anomalies)")

# Visualisation distribution train vs test
```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(X_train["debit_ml_h"], bins=40,
             color="steelblue", alpha=0.7, label="Train")
axes[0].hist(X_test["debit_ml_h"], bins=40,
             color="orange", alpha=0.7, label="Test")
axes[0].set_title("Distribution Débit - Train vs Test")
axes[0].legend()

axes[1].hist(X_train["pression_mmhg"], bins=40,
            color="steelblue", alpha=0.7, label="Train")
axes[1].hist(X_test["pression_mmhg"], bins=40,
            color="orange", alpha=0.7, label="Test")
axes[1].set_title("Distribution Pression - Train vs Test")
axes[1].legend()

plt.tight_layout()
plt.savefig("split_distribution.png", dpi=150, bbox_inches="tight")
plt.show()

```

ÉTAPE 4 - SÉLECTION FEATURES ET SPLIT

Features sélectionnées : ['debit_ml_h', 'pression_mmhg']

X shape : (12096, 2)

y shape : (12096,)

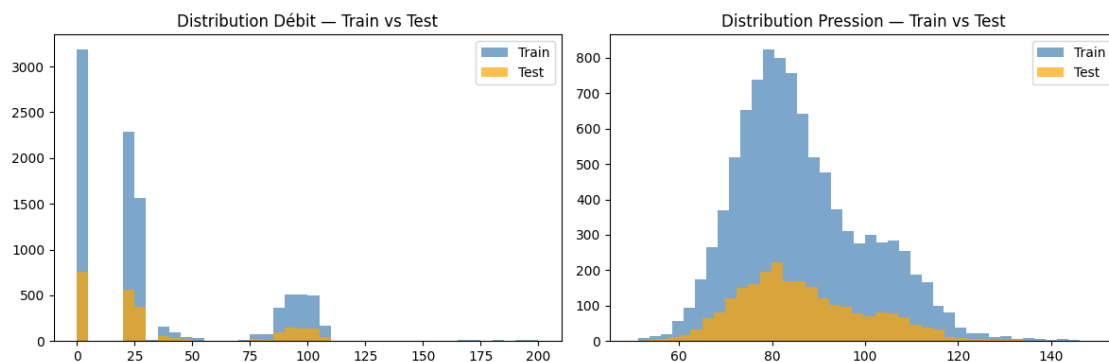
Corrélation entre features :

	debit_ml_h	pression_mmhg
debit_ml_h	1.000000	0.819682
pression_mmhg	0.819682	1.000000

Split Train/Test (80/20) :

Train : 9676 échantillons (86 anomalies)

Test : 2420 échantillons (22 anomalies)



1.0.9 Cellule 9 — Entraînement et résultats finaux

```
[14]: from sklearn.tree import DecisionTreeClassifier
      from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import (accuracy_score, classification_report,
                                   confusion_matrix, ConfusionMatrixDisplay)

      print("=" * 50)
      print("ÉTAPE 5 - ENTRAÎNEMENT ET ÉVALUATION")
      print("=" * 50)

      modeles = {
          "Seuil Simple": None,
          "Régression Logistique": LogisticRegression(max_iter=1000),
          "Arbre de Décision": DecisionTreeClassifier(max_depth=3, random_state=42)
      }

      DEBIT_MIN, DEBIT_MAX = 60, 120
      PRESSION_MIN, PRESSION_MAX = 60, 140

      fig, axes = plt.subplots(1, 3, figsize=(16, 4))

      for i, (nom, modele) in enumerate(modeles.items()):
          if modele is None:
              # Seuil simple
              y_pred = [
                  0 if (DEBIT_MIN <= r["debit_ml_h"] <= DEBIT_MAX and
                      PRESSION_MIN <= r["pression_mmhg"] <= PRESSION_MAX)
                  else 1
                  for _, r in X_test.iterrows()
              ]
          else:
              modele.fit(X_train, y_train)
              y_pred = modele.predict(X_test)

          acc = accuracy_score(y_test, y_pred)
          print(f"\n{'='*40}")
          print(f"  {nom}")
          print(f"{'='*40}")
          print(f"  Accuracy : {acc:.4f}")
          print(classification_report(y_test, y_pred,
                                      target_names=["Normal", "Anomalie"]))

          # Matrice de confusion
          cm = confusion_matrix(y_test, y_pred)
```

```

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Anomalie"]
)
disp.plot(ax=axes[i], colorbar=False, cmap="Blues")
axes[i].set_title(f"{nom}\nAccuracy: {acc:.4f}")

plt.tight_layout()
plt.savefig("matrices_confusion.png", dpi=150, bbox_inches="tight")
plt.show()
print("\n Matrices de confusion sauvegardées")

```

ÉTAPE 5 - ENTRAÎNEMENT ET ÉVALUATION

Seuil Simple

```

Accuracy : 0.2550

```

	precision	recall	f1-score	support
Normal	1.00	0.25	0.40	2398
Anomalie	0.01	1.00	0.02	22
accuracy			0.25	2420
macro avg	0.51	0.62	0.21	2420
weighted avg	0.99	0.25	0.39	2420

Régression Logistique

```

Accuracy : 1.0000

```

	precision	recall	f1-score	support
Normal	1.00	1.00	1.00	2398
Anomalie	1.00	1.00	1.00	22
accuracy			1.00	2420
macro avg	1.00	1.00	1.00	2420
weighted avg	1.00	1.00	1.00	2420

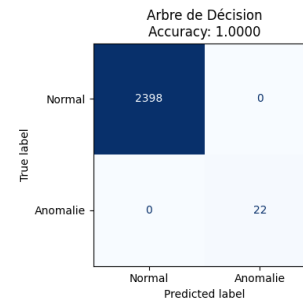
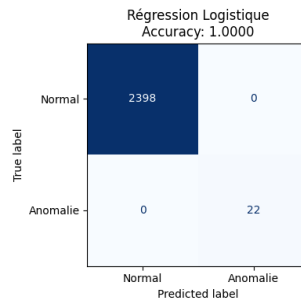
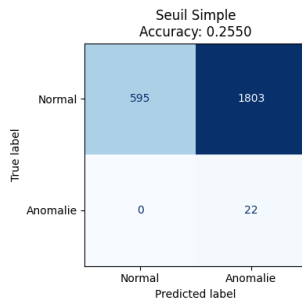
Arbre de Décision

```

Accuracy : 1.0000

```

	precision	recall	f1-score	support
Normal	1.00	1.00	1.00	2398
Anomalie	1.00	1.00	1.00	22
accuracy			1.00	2420
macro avg	1.00	1.00	1.00	2420
weighted avg	1.00	1.00	1.00	2420



Matrices de confusion sauvegardées

1.0.10 Cellule 10 — Résumé final pipeline

```
[15]: print("=" * 60)
print("  RÉSUMÉ PIPELINE COMPLET - GROUPE 6")
print("=" * 60)
print(f"""
DONNÉES BRUTES NAB
    3 fichiers CSV - {len(df_brut)} lignes total

NETTOYAGE
    Suppression doublons      : {doublons_avant} supprimés
    Valeurs manquantes        : 0 trouvées
    Conversion timestamp      :
    Détection outliers        : (IQR x3)

NORMALISATION
    Valeur brute → débit ml/h [0-200]
    Pression mmHg corrélée générée

ÉTIQUETAGE
    Méthode : moyenne ± 2 écarts-types
    Anomalies : {int(df_clean['alerte'].sum())} / {len(df_clean)}
               ({df_clean['alerte'].mean()*100:.1f}%)

```

```

SPLIT TRAIN/TEST
  Train : {len(X_train)} échantillons (80%)
  Test  : {len(X_test)} échantillons (20%)
  Stratified split

MODÈLES ENTRAÎNÉS
  Seuil Simple      :
  Régression Logistique :
  Arbre de Décision  :

""")

```

RÉSUMÉ PIPELINE COMPLET - GROUPE 6

DONNÉES BRUTES NAB

3 fichiers CSV - 12096 lignes total

NETTOYAGE

Suppression doublons : 0 supprimés
 Valeurs manquantes : 0 trouvées
 Conversion timestamp :
 Détection outliers : (IQR x3)

NORMALISATION

Valeur brute → débit ml/h [0-200]
 Pression mmHg corrélée générée

ÉTIQUETAGE

Méthode : moyenne ± 2 écarts-types
 Anomalies : 108 / 12096
 (0.9%)

SPLIT TRAIN/TEST

Train : 9676 échantillons (80%)
 Test : 2420 échantillons (20%)
 Stratified split

MODÈLES ENTRAÎNÉS

Seuil Simple :
 Régression Logistique :
 Arbre de Décision :